NESNE YÖNELİMLİ PROGRAMLAMA

Ders 11: Dinamik Formlar

Konular

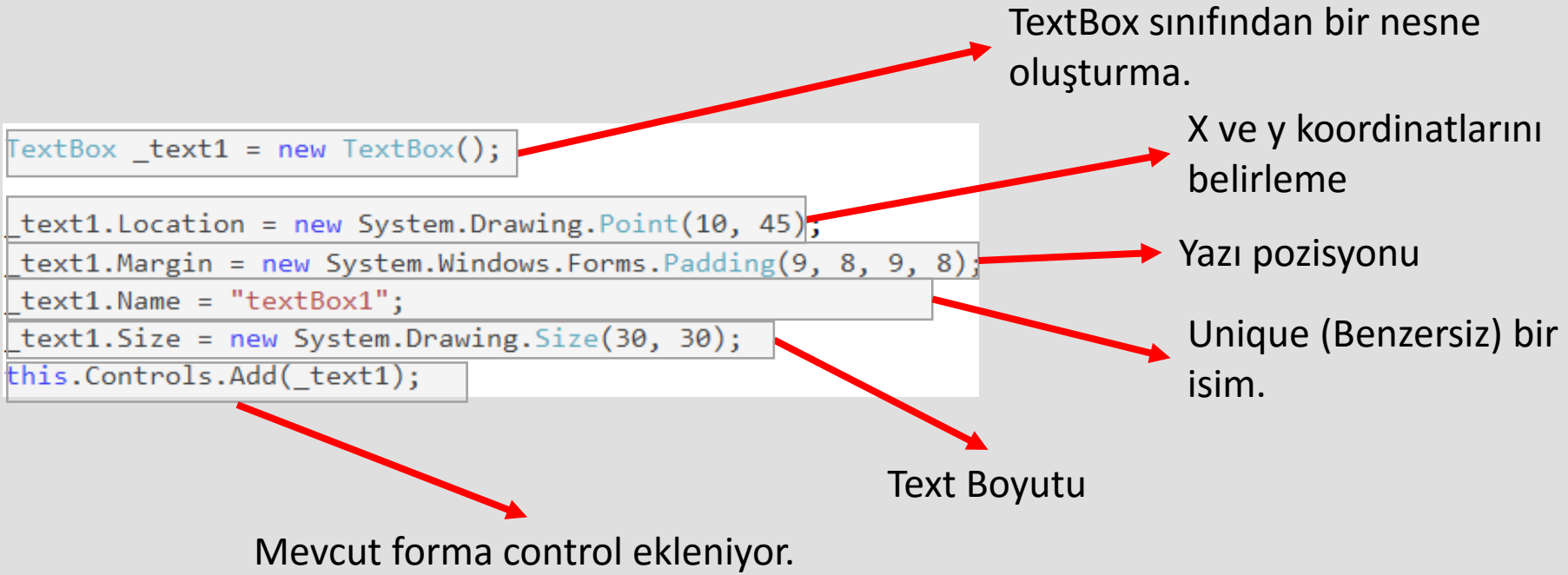
- Dinamik Formlar
  - Dinamik şekilde bileşen oluşturma
  - Sender
  - Event tanımlama
- Karlar Form'da düşer
- Gelişmiş bir Windows Forms Uygulaması

Dinamik Formlar

- Formları ve Form üzerine eklediğiniz tüm Componentleri çalışma anında oluşturabilir, değiştirebilir ve silebilirsiniz.
- Eklenen her component'e dynamic controls denir. Bu bölümde dynamic controls'ün avantajları anlatılacaktır.
- Her ne kadar sürekle-bırak özelliği Windows Forms'ta kod yazmamızı kolaylaştırsa da bazı uygulamalarda süreklem-bırak yetersiz kalmaktadır.
- Örneğin,
  - bir stat, sinema veya tiyatro için izleyici bilet satış uygulaması.
  - Otobüs, uçak uygulamaları.
- 41.903 koltuktan oluşan Vodafone Arena için 41.093 koltuk için sürekle bırak ile button eklemek ve button olaylarını teker teker yazmak imkansız bir süreçtir.
- Benzer şekilde bir otobüs firması için 30 koltuklu, 40 koltuklu otobüs'ler için ayrı ayrı ekranlar tasarlayıp kod yazmak geçerli bir süreç değildir.
- Rastgele karakterlerin oluştuğu oyunlar içinde statik formlar hiç kullanışlı değildir.

Dinamik Formlar

- Nesne oluşturma ve mevcut Form'a ekleme…



- Benzer şekilde Button, Label ve diğer control'ler Form'a eklenebilir.
- Tüm özellikleri ezberlemek zorunda değilsiniz, geçen hafta gördüğümüz InitializeComponent metotundan kopya çekebilirsiniz.
- Bu kod Form_Load event'ına eklenebilir. Form_Load event'i için Form üzerinde bileşen bulunmayan bir yere tıklamanız veya Property->Event iconu ve Form_Load event'ına tek tıklamanız yeterlidir.

Dinamik Formlar

- Amacımız birden fazla nesne oluşturmaktı. Bu noktada Bilgisayar Mantığı devreye girer. Bilgisayar Mantığı ezberle, sınavla veya zorlayarak olacak bir şey değildir. Bu işi seveceksiniz.

```
TextBox _text1 = new TextBox();

_text1.Location = new System.Drawing.Point(10, 45);
_text1.Margin = new System.Windows.Forms.Padding(9, 8, 9, 8);
_text1.Name = "textBox1";
_text1.Size = new System.Drawing.Size(30, 30);
this.Controls.Add(_text1);
```

- Birden fazla nesnem olacak. Her nesnenin koordinatı farklı olsun.
  - Çözüm: Bir veya iki döngü ile bu işi halledebilirim. X ve Y koordinatlarını her nesne için değiştireceğim.
- Her nesnemi ileride yönetmek isteyebilirim. Name özelliği nesneme ait ayırtedici bir özellik olabilir.
  - Çözüm: Döngüden elde edilecek bir değer ile tek ve benzersiz bir name üretebilirim.

Dinamik Formlar

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void Form1_Load(object sender, EventArgs e)
    {
        for (int j = 0; j < 5; j++)
        {
            for (int i = 0; i < 5; i++)
            {
                TextBox _text1 = new TextBox();

                _text1.Location = new System.Drawing.Point(10 + i * 40, 45 + j * 40);
                _text1.Margin = new System.Windows.Forms.Padding(9, 8, 9, 8);
                _text1.Name = "textBox" + i.ToString() + "_" + j.ToString();
                _text1.Size = new System.Drawing.Size(30, 30);
                this.Controls.Add(_text1);
            }
        }
    }
}
```

- X ve Y koordinatları için iki for kullandım. (Tek for ile de yapılabilir. Ben bunu tercih ettim.)
- Pozisyon bilgisini 10 + i * 40, 45+j * 40: X ve Y koordinatlarında 40'lık ötelemeler olacak.
- İsim Bölümü: textBox_0_0 başlayıp hep tek ve benzersiz değer olacak.

Dinamik Formlar - Sender

- Yanyana bir Button ve TextBox olsun. Hangi Button'a tıklanırsa yanındaki TextBox'a Tıklandı yazsın ve diğerlerine hiçbir şey yazmasın.
- Öncelikle Button ve TextBox'ları oluşturalım.

```
private void Form1_Load(object sender, EventArgs e)
{
    for (int i = 0; i < 5; i++)
    {
        Button _btn = new Button();
        _btn.Location = new System.Drawing.Point(60, 25 + i * 35);
        _btn.Name = "btn_" + i.ToString();
        _btn.Size = new System.Drawing.Size(30, 30);
        _btn.Text = "=";
        _btn.Click += new EventHandler(btn_tikla);
        this.Controls.Add(_btn);

        TextBox _txt = new TextBox();

        _txt.Location = new System.Drawing.Point(10, 30 + i * 35);
        _txt.Name = "txt_" + i.ToString();
        _txt.Size = new System.Drawing.Size(50, 30);
        this.Controls.Add(_txt);
    }
}
```

Dinamik Formlar - Sender

- Diğer örneğimizden farklı olarak btn_tikla adında bir metotu Olay olarak bağladık. (Geçen haftaki konuyu hatırlayalım.)

```
_btn.Click += new EventHandler(btn_tikla);
```

- Şimdiki sorumuz hangi Button'dan bir istek geliyor????
  - Bunu ancak ve ancak Sender üzerinden çözebiliriz.
  - Sender Object türündendir. Button_Click event'ında olduğu gibi hem sender hem de EventArgs ekleyelim.
  - Object türünden olduğu için ve olayın bir Button'dan geldiğini bildiğimiz için BİLİNÇLİ DÖNÜŞÜM yapabiliriz. Bilinçli dönüşüm sonucu artık elimizde object değil Tıklama işleminin yapıldığı Button var.

```
private void btn_tikla(object sender, EventArgs e)
{
    Button _btn = (Button)sender;
}
```

Dinamik Formlar

- Nesne oluşturma bölümünde Button ve Text'in Name özelliğine aynı değeri vermiştik. Ekrandaki TextBox'ları dolaşıp i'si uyuşan bölüme tıklandı diğer bölümlere boş string yazdıralım. Name özelliğinden i nasıl çekeceğiz.
  - btn_1
  - btn_2
  - btn_3
- Alt çizgiden sonraki sayıyı al. Sayı 10 geçebilir, buna dikkat et. Evet, yine «Bilgisayar Mantığı»… Bilgisayar Mantığın yoksa bu ortamda yaşayamazsın. İlk bakışta aklıma 3 çözüm geldi.
  - _ bölümünden sonrasını al.
  - 4. karakterden sonrasını al.
  - Sayı buldukça geçici değişkene at.
- Bu noktadan sonra hangi çözüm daha hızlıdır konusunu düşünmeliyiz. (Veri Yapıları dersinin amacıdır.)

Dinamik Formlar

```
#region Farklı sayı Bulma Fonksiyonları
private string sayilari_bul1(string str)
{
    string[] _dizi = str.Split('_'); //karakter tabanlı

    return _dizi[1];
}

private string sayilari_bul2(string str)
{
    return str.Substring(4); //kural tabanlı
}

private string sayilari_bul3(string str) //sayı tabanlı
{
    string temp = "";
    for (int i = 0; i < str.Length; i++)
    {
        try
        {
            int x = Convert.ToInt32(str[i].ToString());
            temp += str[i];
        }
        catch (Exception)
        {
        }
    }

    return temp;
}
#endregion
```

- Split fonksiyonu dizi üretir. Dizinin son elemanı veya ikinci elemanı benim sayımdır.
- Substring ile string'in belli bir pozisyonundan sonrası alınabilir. Ben 4. karakterden sonrasını aldım ve sayıya ulaştım.
- String'in tüm elemanlarını dolaşırım ve Convert.ToInt32 yaparım. Eğer hata veriyorsa Catch bölümüne gider, hiçbir şey yapmaya gerek yok. Eğer sayı dönüşümü yapıldıysa bunu temp bir değişkene eklerim.

- En hızlısın ikinci çözümdür.
- Bu fonksiyonlar ile hem txt hem de çözümlenebilir.

Dinamik Formlar - Controls

- Öncelikle Button'ın sayısını bulduk.
- Form üzerindeki her bir nesneye Controls dizisi içindedir. Controls dizisi bir for döngüsü ile dolaşıyoruz.
- TextBox bulduğumuzda TextBox'ın sayısına bakıyoruz. Eğer sayı eşit ise Text property'sine «Tıklandı» aksi taktirde boş string yazdırıyoruz.
- Fakat, ne yazık ki Form üzerinde tüm control'ler TextBox değil. Bu durumda TextBox olmayan bir kontrole bilinçli dönüşüm uygulandığında sorun çıkar. Bu sorunu aşmak için try-catch çok basit bir çözümdür.

```
private void btn_tikla(object sender, EventArgs e)
{
    Button _btn = (Button)sender;
    string btn_no = sayilari_bul1(_btn.Name);
    for (int i = 0; i < this.Controls.Count; i++)
    {
        try
        {
            TextBox _txt = (TextBox)this.Controls[i];

            if (btn_no == sayilari_bul1(_txt.Name))
                _txt.Text = "Tıklandı";
            else
                _txt.Text = "";
        }
        catch (Exception)
        {

        }
    }
}
```

Karlar Düşer

- LABEL nesneni içine bir tek kar tanesi ile başlayalım.

```
private void Form1_Load(object sender, EventArgs e)
{
    Label LBL_X = new System.Windows.Forms.Label();
    LBL_X.Location = new System.Drawing.Point(10, 10);
    LBL_X.Text = "*";
    LBL_X.AutoSize = true;
    LBL_X.Name = "LBL_1";
    this.Controls.Add(LBL_X);
}
```

Karlar Düşer

- Mevcut Kar tanesi hareket ettirme. Console uygulamasında TimerCallback fonksiyonunu kullanmıştık.
- Windows Forms'ta Timer nesnesinden bir tane oluşturuyorum ve Events bölümünden Tick bölümüne tıklıyorum. Her 100 ms bir yapılacak kodları buraya yazabilir.
- Kar tanesi rüzgar eşliğinde iniyor. Nesneyi buluyorum ve pozisyon bilgisini değiştiriyorum.
- For sayesine Form üzerindeki tüm Label nesnelerine aynı işlem uygulanıyor.

```
private void timer1_Tick(object sender, EventArgs e)
{
    for (int i = 0; i < this.Controls.Count; i++)
    {
        try
        {
            Label LBL_X = (Label)this.Controls[i];
            x += 1;
            y += 1;
            LBL_X.Location = new System.Drawing.Point(x, y);
        }
        catch
        {
        }
    }
}
```

Karlar Düşer

- Çalışma anında birçok kar tanesi olacak. Her birinin ayrı bir Location olacak. Bu durumda bir nesne tanımlayalım.
- Bir tek kar tanesi için bir nesne yarattık.
  - Constructor ile kar tanesi bilgisini alıyor.
  - Kar_dus metodu y koordinatında kar tanesini bir adım öteliyor.
  - X ve Y property'leri ile de x ve y değerleri görüntüleniyor. Encapsulation kuralı uygun oldu.
  - Kar_dus fonksiyonu ile kar düşüş hızını ayarlayabiliriz. (Teker teker düşürmek yavaş olabilir.)

```
public class KarTanesi
{
    private int x, y;

    public KarTanesi(int _x, int _y)
    {
        x = _x;
        y = _y;
    }

    public void kar_dus()
    {
        y++;
    }

    public int X
    {
        get { return x; }
    }

    public int Y
    {
        get { return y; }
    }
}
```

Karlar Düşer

- Birden fazla kar tanemiz olacak ve bu kar taneleri yöneteceğiz. List özelliği ile kar tanelerimizi saklayabiliriz.
- KarTanesiYarat metodu x ve y değerlerine göre bir Label yaratsın.
- Form_Load bölümüne Kar tanesi listemizi yaratıyoruz. Artık Listemize ekleme yapabiliriz.

```
List<KarTanesi> Kar_List;

public Form1()
{
    InitializeComponent();
}

private void KarTanesiYarat(int x, int y)
{
    Label LBL_X = new System.Windows.Forms.Label();
    LBL_X.Location = new System.Drawing.Point(x, y);
    LBL_X.Text = "*";
    LBL_X.AutoSize = true;
    LBL_X.Name = "LBL_1";
    this.Controls.Add(LBL_X);
}

private void Form1_Load(object sender, EventArgs e)
{
    Kar_List = new List<KarTanesi>();//Liste hazır
}
```

Karlar Düşer

- y=1 koordinatında ve x rastgele bir sayı koordinatına bir kar tanesi yaratılsın.
- Form'un dışına kar tanesi gitmesin diye This.Width ile Form'un genişliği bulunuyor.
- Ekleme işlemi Kar_List değişkeni içine yapılıyor.

```
//istenildiği kadar kar tanesi yaratır.
private void RastgeleKarTaneleriYarat(int sayi)
{
    Random _r = new Random();
    for (int i = 0; i < sayi; i++)
    {
        int x = _r.Next(this.Width);
        KarTanesiYarat(x, 1);
        Kar_List.Add(new KarTanesi(x, 1));
    }
}
```

Karlar Düşer

- Kar tanelerini Form Height'ının dışına çıkarsa hem Kar_List'ten hem de Form'dan silelim ki boşu boşuna işlem yapmasın. Bu durumda for döngüsü yerine sonsuz bir while döngüsü açmak daha mantıklıdır. Çünkü bir silme işlemi olunca i değeri aynı kalmalıdır. For ise i değerini hep arttırır, bunu önlemek için while döngüsü kullanılmıştır.
- RastgekeKarTaneleriYarat fonksiyonu sayesinde istenildiği kadar kar tanesi başlangıçta yaratılır.

```
private void timer1_Tick(object sender, EventArgs e)
{
    RastgeleKarTaneleriYarat(5);
    int i = 0;
    while (true)
    {
        try
        {
             if (i == this.Controls.Count)
                break;

            KarTanesi _kt = Kar_List[i];
            _kt.kar_dus();
            Label LBL_X = (Label)this.Controls[i];

            if (_kt.Y >= this.Height)
            {
                Kar_List.RemoveAt(i);
                this.Controls.Remove(LBL_X);
                LBL_X = null;
            }
            else
            {
                LBL_X.Location = new System.Drawing.Point(_kt.X, _kt.Y);
                Kar_List[i] = _kt;
                i++;
            }

        }
        catch
        {
        }
    }
}
```

Karlar Düşer

- Kar tanesi bilgisi sınıftan ve Label bilgisi de i.inci kontrolden alınır.
- Eğer form'un Height bilgisi aşılmızsa List türünden Kar_List listesinden RemoveAt komutu ile çıkarma işlemi yapılır. Form üzerinden çıkarma işlemi için ise Controls.Remove nesne bilgileri kullanılabilir ve mevcut nesne null yapılarak GC'ye bırakılır.
- Eğer form'un height değeri aşılmadıysa direkt seçili Label'ın koordinat değerleri ve Kar_List listesindeki güncelleme yapılıyor. İ değeri de sonraki kar tanesine geçmek için kullanılıyor.

```
private void timer1_Tick(object sender, EventArgs e)
{
    RastgeleKarTaneleriYarat(5);
    int i = 0;
    while (true)
    {
        try
        {
             if (i == this.Controls.Count)
                break;

            KarTanesi _kt = Kar_List[i];
            _kt.kar_dus();
            Label LBL_X = (Label)this.Controls[i];

            if (_kt.Y >= this.Height)
            {
                Kar_List.RemoveAt(i);
                this.Controls.Remove(LBL_X);
                LBL_X = null;
            }
            else
            {
                LBL_X.Location = new System.Drawing.Point(_kt.X, _kt.Y);
                Kar_List[i] = _kt;
                i++;
            }

        }
        catch
        {
        }
    }
}
```

Gelişmiş bir Windows Forms Uygulaması

- Bu uygulamanın amacı NYP ile Windows Forms etkileşimli bir uygulama geliştirmek olacaktır.
- Uygulama
  - Bir öğrenci sınıfı oluşturun.
  - Aynı sınıfa ait bir List oluşturun. List'inde kendine ait bir sınıfı ve dosyası olsun. (Bu iki sınıf Windows Forms projesinden ayrı bir projede (Library Projesi) olarak olsun.)
  - List üzerine ekleme, silme ve düzeltme işlemlerini yapan kodu yazın.
    - Ekleme ve düzeltme işlemleri için yeni bir form açılsın. Kaydetme veya kaydetme işlemi yapmadan çık olsun, yani Form kapatılsın.
  - List içinde Öğrenci arama olsun.

Derste yapılan uygulamayı ADYS üzerinden inceleyebilirsiniz. Bu uygulamadaki amaç büyük projelerde projeyi sınıflara bölme ve sınıfları kullanma üzerine olacaktır. Sınıf bir Console uygulamasından, bu örnekte de olduğu bir Windows Forms uygulamasından veya ileri de bir web uygulamasından dahi kullanılabilecek şekilde tasarlanacaktır.

Kaynaklar

- Google
- stackoverflow